



UNIVERSIDADE FEDERAL DE MATO GROSSO
CAMPUS UNIVERSITÁRIO DE VÁRZEA GRANDE
FACULDADE DE ENGENHARIA
ENGENHARIA DE COMPUTAÇÃO

MODELAGEM GRÁFICA
PROF. FABRÍCIO CARVALHO

Trabalho Prático

Luciano Tadeu, Samuel Borges

1 Descrição da cena

A aplicação consiste em uma cena tridimensional interativa que simula um sistema estelar. O ambiente é composto por um Sol central que emite luz, cercado por diversos corpos celestes em órbita ou em posições estáticas, além de um foguete espacial animado. A cena é delimitada por um "céu estrelado" procedural (skybox de pontos) e um plano quadriculado na base. O usuário atua como um observador livre (câmera em primeira pessoa) capaz de navegar pelo espaço sideral, interagindo com a física básica de pulo e com a colisão do Sol, enquanto um HUD bidimensional fornece o estado em tempo real das variáveis do sistema.

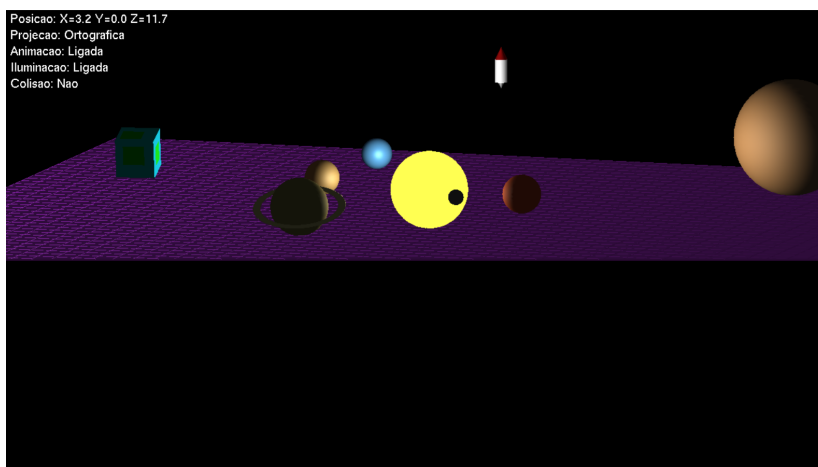


Figura 1: Aplicação em Funcionamento

2 Objetos utilizados

A construção geométrica da cena fez uso de diversas primitivas da biblioteca FreeGLUT e OpenGL padrão:

- **Esferas** (`glutSolidSphere`): Utilizadas para modelar o Sol e os planetas estáticos Mercúrio, Vênus, Júpiter, Marte, Urano e a Lua.
- **Cubos** (`glutSolidCube`): Aplicados na geração do piso base da cena e na estrutura principal do Planeta Terra.
- **Primitivas Diretas** (`GL_QUADS` e `GL_LINES`): Utilizadas para desenhar os continentes decalquados nas faces da Terra cúbica e para desenhar a grade tridimensional (Grid) do cenário.
- **Torus** (`glutSolidTorus`): Utilizado em conjunto com transformações de escala para desenhar os anéis achatados de Saturno.
- **Cilindros e Cones** (`glutSolidCylinder`, `glutSolidCone`): Combinados para formar o modelo estrutural do foguete (corpo, bico e motor).
- **Pontos** (`GL_POINTS`): Utilizados na geração de 1.000 estrelas calculadas via coordenadas esféricas polares para compor o fundo sideral.

O projeto atende ao requisito de objetos compostos através do Sistema Terra-Lua (cubo com decalques e satélite), Saturno (esfera sobreposta a um torus) e o Foguete (cilindro central com cones nas extremidades).

3 Transformações aplicadas

A modelagem hierárquica foi intensamente explorada através do uso das pilhas de matrizes (`glPushMatrix` e `glPopMatrix`). As transformações afins aplicadas incluem:

- **Translação** (`glTranslatef`): Utilizada para posicionar os corpos no espaço e para calcular o raio das órbitas ao redor do Sol usando trigonometria (`cos` e `sin`).
- **Rotação** (`glRotatef`): Aplicada para o giro dos planetas em torno de seus próprios eixos e para a órbita dinâmica. Em objetos como Saturno, foi utilizada uma rotação dupla para inclinar os anéis e gerar um efeito de balanço (*wobble*).
- **Escala** (`glScalef`): Utilizada para redimensionar o piso, achatar drasticamente o eixo Y do anel de Saturno e para criar um efeito de pulsação orgânica no foguete, cujo fator de escala varia com base em uma função senoidal.

4 Funcionamento da câmera

A câmera implementa um sistema de navegação em primeira pessoa (FPS) fluido. A visualização é calculada pela função `gluLookAt`. O controle de visão (Yaw e Pitch) é ditado pelo movimento passivo do mouse (`glutPassiveMotionFunc`), que altera a direção do vetor de visão (*Look At*) utilizando trigonometria esférica, além de aplicar uma trava (*Gimbal Lock*) para impedir que a câmera vire de cabeça para baixo.

A movimentação direcional ocorre pelas teclas W, A, S e D, que alteram as coordenadas da câmera em relação ao vetor de direção atual. Também foi implementada uma simulação cinemática simples ativada pela tecla Espaço, que atribui uma velocidade vertical à câmera sujeita a uma constante de gravidade, resultando em um pulo parabólico.

5 Funcionamento da projeção

A aplicação permite alternar dinamicamente o volume de visualização (Frustum) da câmera através da tecla 'P'.

- **Projeção Perspectiva:** Padrão do sistema, executada via `gluPerspective` com um campo de visão (FOV) de 75 graus, garantindo profundidade realista.
- **Projeção Ortográfica:** Executada via `glOrtho`, a ausência de perspectiva de fuga simula uma visualização geométrica paralela (isométrica).

O sistema conta ainda com uma segunda matriz de projeção ortográfica puramente bidimensional (`gluOrtho2D`), temporariamente ativada na rotina de desenho para renderizar os textos do HUD colados diretamente na tela (espaço de janela), independentemente de onde a câmera 3D esteja apontando.

6 Implementação da iluminação

O modelo de iluminação de Phong foi implementado utilizando uma única fonte de luz posicional (`GL_LIGHT0`) atrelada às coordenadas do objeto do Sol. Os materiais dos objetos foram meticulosamente configurados com diferentes coeficientes de `GL_AMBIENT`, `GL_DIFFUSE`, `GL_SPECULAR` e `GL_SHININESS` para simular texturas distintas: a opacidade pulverulenta de Marte e da Lua, o brilho cristalino da Terra e o reflexo metálico polido do foguete. O corpo principal do Sol possui uma propriedade de `GL_EMISSION`, garantindo seu brilho contínuo independentemente da luz. O cálculo de normais (`glNormal3f`) foi configurado manualmente para os polígonos não-primitivos (continentes da Terra).

7 Estratégia de colisão

A lógica de colisão implementa a técnica de Resolução Contínua de Colisão através de uma esfera delimitadora (Bounding Sphere). Na rotina de atualização (`timer`), a Distância Euclidiana em 3D entre as coordenadas centrais da câmera e as do Sol é calculada a cada quadro. Se essa distância for igual ou inferior ao raio de colisão estipulado, o sistema extrai o vetor normalizado da direção do impacto e redefine imediatamente a posição do observador para a borda exata da área de restrição. Visualmente, isso barra o avanço do jogador e atualiza o estado do HUD, que sinaliza o impacto mudando sua cor para vermelho através de `glColor3f(1.0, 0.0, 0.0)`.

8 Funcionamento da animação

A atualização lógica do sistema é controlada por uma máquina de estados de tempo (`glutTimerFunc`) que executa uma rotina a aproximadamente 60 quadros por segundo (16 ms). Quando ativada (tecla 'T'), as variáveis de estado angulares globais são

incrementadas continuamente. Essas variáveis são consumidas nas chamadas de transformações matriciais na rotina de renderização (`display`), criando translações orbitais complexas. A abordagem de desvincular a lógica de atualização (Timer) da lógica de desenho (Display) permite que elementos estáticos coexistam com as animações sem prejudicar a taxa de renderização e mantém a estabilidade do HUD.